

System Architecture & Engineering Design

AI-Powered Project Intelligence and Knowledge Management Platform

*Transitioning from Communication-First Tools to Evolving Knowledge
Systems*

Project Design & Architecture Team

July 6, 2026

Abstract

This document outlines the system architecture, product specification, and detailed engineering design for an AI-native collaborative workspace. The platform continuously converts live conversations, video meetings, and project metadata into a structured, searchable, and actionable project knowledge memory. Moving away from standard communication-first models (e.g., video conferencing clones), this platform establishes a centralized "Project Intelligence" framework.

We describe a seven-layer conceptual architecture spanning collaboration, audio capture, structured knowledge synthesis, AI enrichment, storage, semantic reasoning, and action execution. Furthermore, we define the concrete technical choices made for the V1 implementation: including the integration of LiveKit for WebRTC infrastructure, SuperTokens for authentication, pgvector for integrated vector/relational databases, FastAPI as a modular-monolith Gateway/BFF, and an event-driven background worker pipeline powered by Redis Streams. Finally, folder structures, database schemas, message queues, and API contracts are specified to provide a comprehensive engineering blueprint.

Contents

1	Product Vision and Core Philosophy	3
1.1	Introduction: The Paradigm Shift	3
1.2	The Core Problem Addressed	3
1.3	Evolving from Read-Only to Action-Oriented	4
2	Conceptual System Architecture	5
2.1	1. Collaboration Layer	6
2.2	2. Capture Layer	6
2.3	3. Knowledge Layer	6
2.4	4. Intelligence Layer	6
2.5	5. Knowledge Repository	6
2.6	6. Reasoning Layer	7
2.7	7. Action Layer	7
3	Technology Stack Decisions and Trade-offs	8
3.1	Identity Management: SuperTokens vs. Keycloak	8
3.2	Vector Databases: pgvector vs. Qdrant	8
3.3	WebRTC Infrastructure: LiveKit vs. Custom SFU	9
3.4	Voice Activity Detection (VAD) Strategy	9
3.5	Unified Technical Stack Specification	9
4	Database Design and Event Model	11
4.1	Logical Domains	11
4.2	Entity Relationship Diagram (Conceptual)	11
4.3	Relational Schema Specification	11
4.4	Object Storage (MinIO) Bucket Layout	12
4.5	Redis Events Architecture	13
5	Frontend Engineering	14
5.1	Feature-First Directory Layout	14
5.2	State Architecture: Zustand + TanStack Query	14
5.3	API Client and Abstraction Contract	15
5.4	Dual-WebSocket Strategy	15
6	Backend Engineering	16
6.1	The Modular Monolith Strategy	16
6.2	Backend-for-Frontend (BFF) Layer	16
6.3	Execution Contracts: Layer Separation	17

7	Roadmap and System Specs Blueprint	18
7.1	System Blueprints	18
7.2	Evolutionary Roadmap	18

Chapter 1

Product Vision and Core Philosophy

1.1 Introduction: The Paradigm Shift

Historically, collaborative software has separated communication from knowledge repository management. Teams speak in video conferencing applications (e.g., Zoom, Microsoft Teams), discuss tasks in messaging systems (e.g., Slack), manage tickets in project management tools (e.g., Jira), and document findings in wikis (e.g., Notion, Confluence). This dispersion of information results in significant retrieval overhead, key decision amnesia, and dependency on individual memory.

This proposal represents a paradigm shift. Rather than constructing a communication-first "Zoom clone" with supplementary AI features, this platform is an **AI-native project intelligence workspace**. Video meetings and text chats are no longer isolated products; they are ingestion pipelines that feed a unified, evolving project memory.

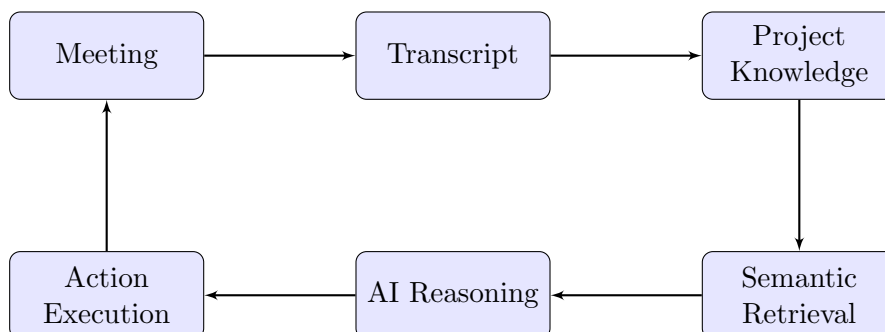


Figure 1.1: The Continuous Knowledge Cycle: Transforming communication events into structured, actionable memory.

1.2 The Core Problem Addressed

Within software engineering teams, knowledge fragments rapidly. Developers frequently ask historical questions such as:

- *"Why did we move away from JWT for session management?"*
- *"Who decided PostgreSQL was superior to a NoSQL option for this module?"*
- *"When did we agree to migrate the auth service, and what were the caveats?"*

Finding these answers requires manually searching Slack history, opening outdated design documents, browsing GitHub PR reviews, or scrolling through hours of meeting recordings. By

structuring conversations and identifying semantic relationships, this platform converts volatile communication back into structured knowledge, making retrieval independent of personal recall.

1.3 Evolving from Read-Only to Action-Oriented

Traditional transcript search engines are passive and read-only. A user asks a question, and the system matches text. This platform introduces an **Action Layer**. By reasoning over meeting context, the AI assistant identifies commitments and executes administrative actions directly:

- Constructing and assigning Jira or GitHub issues.
- Scheduling follow-up meetings via Google Calendar.
- Generating document templates or updating project wiki spaces.
- Sending targeted reminders to project members.

Chapter 2

Conceptual System Architecture

The system is structured into seven distinct architectural layers, ensuring modularity, scalability, and ease of future migration from a modular monolith to distributed microservices.

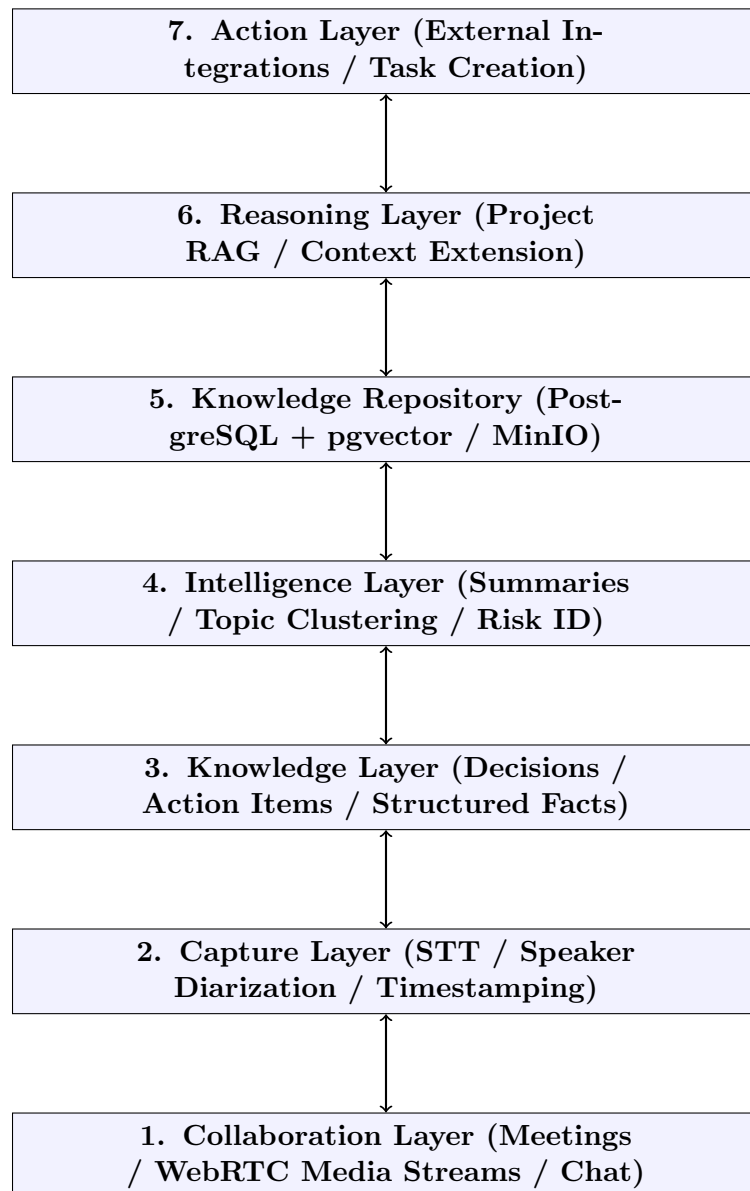


Figure 2.1: The Seven-Layer Conceptual Architecture of the Project Intelligence Platform.

2.1 1. Collaboration Layer

The origin of project knowledge. It manages user interactions including video conferencing (WebRTC), real-time chat, meeting scheduling, and screen sharing. It operates as an event source, generating media streams and client state changes.

2.2 2. Capture Layer

Transforms unstructured binary media streams into text. It performs:

- Real-time speech-to-text (STT) conversion.
- Speaker diarization (identifying "who spoke when").
- Precise word-level timestamping.
- Transcript cleanup (removing stutters, fillers, and formatting text).

2.3 3. Knowledge Layer

Parses the raw transcript text to extract structured semantic facts. Instead of storing long paragraphs, it isolates atomic facts:

- **Decisions:** e.g., *"Use OAuth instead of JWT for session tokens."*
- **Concerns:** e.g., *"Redis memory usage may grow exponentially under high load."*
- **Requirements:** e.g., *"Offline support is required before the public beta release."*
- **Action Items:** e.g., *"John will migrate the auth repository by Friday."*

2.4 4. Intelligence Layer

Applies machine learning models to enrich the extracted facts. Responsibilities include clustering topics, identifying technical entities (e.g., library names, environment variables), assessing risks, and constructing comprehensive meeting summaries.

2.5 5. Knowledge Repository

The persistent memory database. It stores structured transcripts, summaries, topics, and relationships. Critically, it scopes knowledge inside isolated project boundaries to preserve data permissions. Each chunk stores:

```
1 {  
2   id: UUID,  
3   project_id: UUID,  
4   meeting_id: UUID,  
5   speaker: VARCHAR,  
6   chunk_text: TEXT,  
7   embedding: VECTOR(1024), -- BGE-M3 vector dimension  
8   topic: VARCHAR,  
9   created_at: TIMESTAMP  
10 }
```

2.6 6. Reasoning Layer

The retrieval engine (RAG). When a user asks a question, this layer translates the intent, queries the repository, reranks candidate documents, builds a grounded context, and generates a structured answer backed by strict citations.

2.7 7. Action Layer

The agentic execution engine. It interfaces with external platforms (GitHub, Jira, Slack, Google Calendar) to convert text decisions into operational actions, transforming the system from a passive search engine into an active project assistant.

Chapter 3

Technology Stack Decisions and Trade-offs

3.1 Identity Management: SuperTokens vs. Keycloak

For the V1 architecture, SuperTokens was chosen over Keycloak to minimize operational complexity while maintaining high security.

Table 3.1: Identity Management Technology Comparison

Feature	SuperTokens	Keycloak
Open Source	Yes (Apache 2.0)	Yes (Apache 2.0)
Self-hosted	Yes	Yes
Resource Footprint	Low (< 100MB RAM)	High (> 1GB RAM, JVM-based)
Organizations / Tenants	Basic / Code-driven	Excellent / UI-driven
Session Management	Advanced (Automatic rotation)	Standard OAuth2/OIDC
Learning Curve	Easy / Developer-focused	Steep / Enterprise IAM focus
Deployment	Single binary / Docker	WildFly/Quarkus container

Keycloak is an enterprise-grade Identity and Access Management (IAM) suite. However, it imposes significant deployment overhead. SuperTokens provides native libraries for FastAPI and React, custom session rotation mechanisms, lightweight memory usage, and OAuth/OIDC capabilities without requiring complex configuration.

3.2 Vector Databases: pgvector vs. Qdrant

For vector searching, pgvector (integrated into PostgreSQL) is selected over Qdrant.

Key Advantages of pgvector

1. **Single Database Paradigm:** Using a single database engine simplifies backup schedules, replication, migration management, and configuration.
2. **Data Consistency:** Avoiding two databases eliminates the need for sync pipelines between PostgreSQL (metadata) and Qdrant (vectors). A deletion of a project or meeting cascadingly removes vectors immediately.
3. **Relational Joins:** Sub-filtering vectors by metadata (e.g., project permissions, meeting date ranges) is a native SQL join, preventing "two-hop" query overhead.

```

1 SELECT tc.id, tc.chunk_text, m.date, s.name AS speaker_name,
2        1 - (tc.embedding <=> :query_embedding) AS similarity
3 FROM transcript_chunks tc
4 JOIN meetings m ON tc.meeting_id = m.id
5 JOIN speakers s ON tc.speaker_id = s.id
6 WHERE tc.project_id = :project_id
7       AND m.date >= :start_date
8 ORDER BY tc.embedding <=> :query_embedding
9 LIMIT 10;

```

Listing 3.1: Single-query retrieval with pgvector

What is Sacrificed

Qdrant provides superior indexing latency, HNSW search performance at scale (millions of vectors), and built-in advanced reranking integrations. However, for a single-workspace startup V1 with under 100,000 document chunks, pgvector’s HNSW indexes perform under 5ms, making a separate vector engine an unnecessary operational burden.

3.3 WebRTC Infrastructure: LiveKit vs. Custom SFU

Building WebRTC infrastructure from scratch is a massive engineering undertaking. It requires handling Interactive Connectivity Establishment (ICE), STUN/TURN servers, Selective Forwarding Units (SFUs), bandwidth estimation, jitter buffers, and simulcasting.

For this project, **LiveKit** is treated as foundational infrastructure (similar to PostgreSQL or Redis). It manages WebRTC connections, audio/video routing, and media recording. The FastAPI Meeting Service acts as the orchestrator, managing room states, token generation, and linking rooms to specific projects, allowing developers to focus on the downstream AI services.

3.4 Voice Activity Detection (VAD) Strategy

To optimize the real-time transcription pipeline, we drop heavy standalone machine learning models (like Silero VAD) in favor of a hybrid approach:

- **Client-side WebRTC Activity:** LiveKit’s client SDK detects active speaker states before audio packets are sent to the network.
- **Whisper no-speech probability:** The transcription worker checks faster-whisper’s internal `no_speech_prob` score to quickly filter out silent or noise-only packets.

This eliminates pipeline bottlenecks and saves GPU/CPU resources on the server.

3.5 Unified Technical Stack Specification

- **Reasoning LLM:** Qwen 3 (8B–14B parameters depending on host server hardware constraints).
- **Embeddings Engine:** BGE-M3 (dense, sparse, and multi-vector support; 1024 dimensions).
- **Reranking Engine:** BGE-Reranker-Large.
- **Speech-to-Text:** faster-whisper Large-v3.
- **Diarization Engine:** pyannote.audio.

- **Relational/Vector Storage:** PostgreSQL 16 + pgvector 0.5.
- **Object Storage:** MinIO (S3-compatible API).
- **Cache & Event Broker:** Redis 7 (Streams + Pub/Sub).

Chapter 4

Database Design and Event Model

4.1 Logical Domains

The relational database is separated into four distinct business domains:

- **Identity:** Users, Profiles, OAuth accounts.
- **Collaboration:** Projects, Members, Meetings, Participants, Invitations.
- **Knowledge:** TranscriptSegments, KnowledgeChunks, Embeddings, Summaries, Topics, Decisions, ActionItems.
- **AI:** ChatSessions, ChatMessages, SearchLogs, Prompts, Feedback.

4.2 Entity Relationship Diagram (Conceptual)

The platform models all data around a **Project** root:

Project

- Members (Users + Roles)
- Meetings
 - Participants
 - TranscriptSegments
 - Summaries
 - Recordings (Stored in MinIO)
- Decisions
- Action Items
- Knowledge Chunks (with Vectors)

4.3 Relational Schema Specification

Below are the critical schemas written in PostgreSQL DDL syntax.

```
1 CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
2 CREATE EXTENSION IF NOT EXISTS vector;
3
4 -- Projects Table
5 CREATE TABLE projects (
6     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
7     name VARCHAR(255) NOT NULL,
8     description TEXT,
```

```

9      created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
10     updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
11 );
12
13 -- Project Members Table (RBAC)
14 CREATE TABLE members (
15     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
16     project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
17     user_id VARCHAR(255) NOT NULL, -- SuperTokens User ID
18     role VARCHAR(50) CHECK (role IN ('owner', 'admin', 'member')) DEFAULT '
member',
19     joined_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
20     UNIQUE(project_id, user_id)
21 );
22
23 -- Meetings Table
24 CREATE TABLE meetings (
25     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
26     project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
27     title VARCHAR(255) NOT NULL,
28     status VARCHAR(50) CHECK (status IN ('scheduled', 'live', 'completed'))
DEFAULT 'scheduled',
29     livekit_room VARCHAR(255) UNIQUE,
30     recording_url VARCHAR(512),
31     started_at TIMESTAMP WITH TIME ZONE,
32     ended_at TIMESTAMP WITH TIME ZONE,
33     created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
34 );
35
36 -- Transcript Segments Table
37 CREATE TABLE transcript_segments (
38     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
39     meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
40     speaker_id VARCHAR(255) NOT NULL,
41     start_time NUMERIC(8,2) NOT NULL,
42     end_time NUMERIC(8,2) NOT NULL,
43     text TEXT NOT NULL,
44     created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
45 );
46
47 -- Knowledge Chunks Table for pgvector RAG
48 CREATE TABLE knowledge_chunks (
49     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
50     project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
51     meeting_id UUID REFERENCES meetings(id) ON DELETE SET NULL,
52     speaker VARCHAR(255),
53     chunk_text TEXT NOT NULL,
54     embedding VECTOR(1024), -- Dimension of BGE-M3
55     topic VARCHAR(100),
56     created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
57 );
58
59 -- Index for HNSW Cosine Similarity Search
60 CREATE INDEX ON knowledge_chunks
61 USING hnsw (embedding vector_cosine_ops)
62 WITH (m = 16, ef_construction = 64);

```

Listing 4.1: Core Relational Tables

4.4 Object Storage (MinIO) Bucket Layout

Binary objects must never reside in PostgreSQL. MinIO holds all media, structured as follows:

- `recordings/{meeting_id}.mp4` - High quality room recording.
- `audio/{meeting_id}.wav` - Extracted mono channel wav for speech processing.
- `attachments/{project_id}/{file_uuid}.pdf` - Documents ingested for project memory extension.

4.5 Redis Events Architecture

The system uses Redis 7 Streams for asynchronous background worker coordination. Communication is fully event-driven, following the cycle in Figure 4.1.

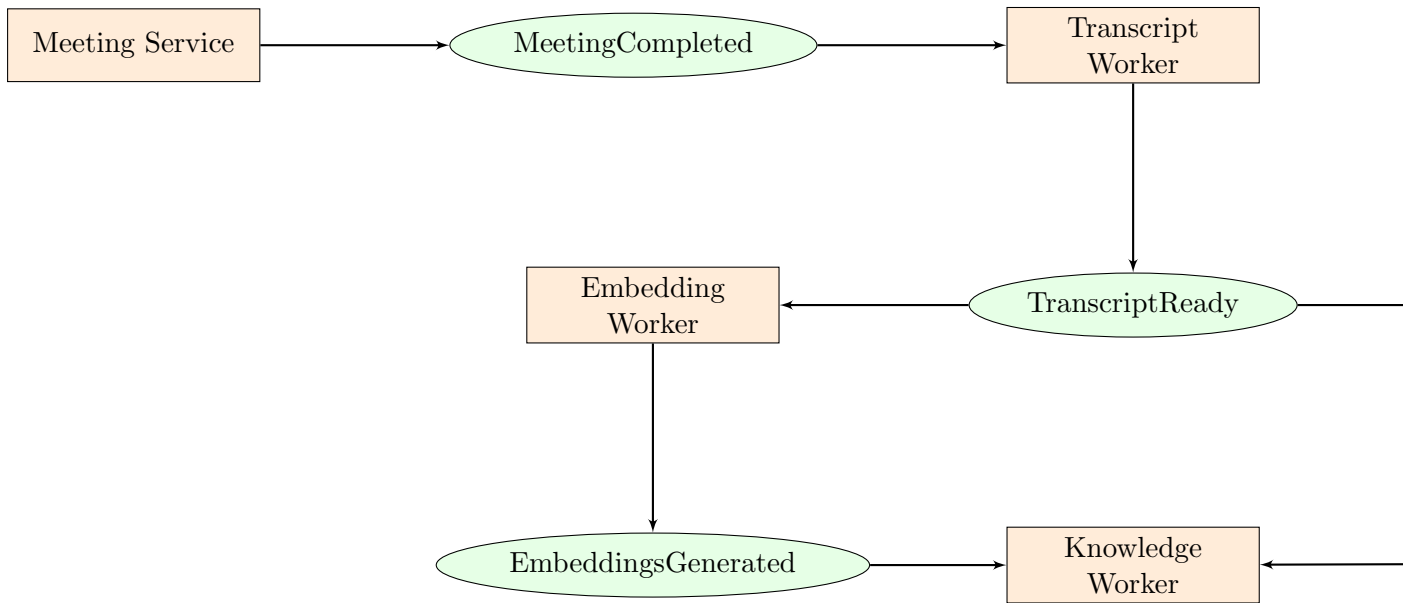


Figure 4.1: Redis Event-Driven Worker Pipeline.

Background Workers Definition

- **Transcript Worker:** Consumes `MeetingCompleted`. Extracts audio from MinIO, invokes pyannote for diarization, whisper for transcription, and writes segments to the database. Emits `TranscriptReady`.
- **Embedding Worker:** Consumes `TranscriptReady`. Splits text into chunks, generates BGE-M3 vectors, and saves them in `knowledge_chunks`. Emits `EmbeddingsGenerated`.
- **Summary Worker:** Consumes `TranscriptReady`. Calls the LLM to generate sections (executive summary, decisions, action points). Writes summaries to the DB and emits `SummaryGenerated`.
- **Knowledge Worker:** Consumes `SummaryGenerated` and `EmbeddingsGenerated`. Synthesizes the data, updates the search cache, and broadcasts notifications. Emits `KnowledgeUpdated`.

Chapter 5

Frontend Engineering

5.1 Feature-First Directory Layout

For scalability, code is arranged into encapsulated business features rather than folder types.

```
src/
  app/                                # Application wrappers, Router, Providers
    router.tsx
    App.tsx
    providers.tsx
  features/                           # Business Domains
    auth/                             # Login, OAuth handlers, SuperTokens wrappers
    projects/                         # Project lists, configuration, details
    meetings/                         # WebRTC client interface, control overlays
    transcript/                       # Real-time scrolling transcript view
    chat/                             # Project RAG assistant chat interface
    knowledge/                       # Extracted topics, decisions dashboard
    dashboard/                       # Global overview page
  components/                         # Shared UI components (Button, Input, Card)
  hooks/                             # Global generic hooks (useLocalStorage, etc.)
  services/                          # Core API layer & WebSocket managers
  stores/                             # Zustand global state declarations
  types/                             # Shared TypeScript structures
```

5.2 State Architecture: Zustand + TanStack Query

To prevent state mud, the system implements a strict boundary:

- **TanStack Query:** Owns all server-state cache. Anything fetched via REST endpoints (projects lists, past transcripts, metadata) is kept here. Query invalidation handles reloading.
- **Zustand:** Owns client-only transient state. No REST data resides in Zustand. It handles: the active user details, the currently active project ID, selected application theme, WebSocket connection status, and active local audio/video stream parameters.

5.3 API Client and Abstraction Contract

Components must never execute HTTP queries or fetch requests directly. To maintain modularity, the architecture requires an abstraction barrier:

UI Component $\longrightarrow$ Custom React Hook $\longrightarrow$ API Service Layer $\longrightarrow$ Axios Instance

```

1 // features/projects/services/projectApi.ts
2 import { apiClient } from '@lib/api';
3 import { Project } from '../types';
4
5 export const projectApi = {
6   getProjects: async (): Promise<Project[]> => {
7     const response = await apiClient.get('/projects');
8     return response.data.data;
9   },
10  createProject: async (name: string, desc?: string): Promise<Project> => {
11    const response = await apiClient.post('/projects', { name, description:
12      desc });
13    return response.data.data;
14  }
15 };
16
17 // features/projects/hooks/useProjects.ts
18 import { useQuery } from '@tanstack/react-query';
19 import { projectApi } from '../services/projectApi';
20
21 export const useProjects = () => {
22   return useQuery({
23     queryKey: ['projects'],
24     queryFn: projectApi.getProjects
25   });
26 };

```

Listing 5.1: Frontend Hook-Service abstraction pattern

5.4 Dual-WebSocket Strategy

The client manages two separate WebSocket systems:

1. **LiveKit SDK WebSocket:** Transports media tracks (audio, video, screen shares) and manages standard WebRTC renegotiation and network state restoration.
2. **Application WebSocket (FastAPI):** Transports light real-time data: live transcript scrolling tokens, background processing milestones, and streaming AI chat tokens.

Chapter 6

Backend Engineering

6.1 The Modular Monolith Strategy

For V1, deploying distributed microservices introduces excessive networking and DevOps overhead. Instead, we use a **Modular Monolith**. Services are written inside strictly isolated domain folders in a single repository. They communicate via event queues (Redis Streams) or explicit Python service interface calls. This architecture allows extracting any service into a standalone container with zero code rewrites.

```
backend/
  api/                                # BFF (Backend-for-Frontend) Gateway Layer
    auth/                             # Auth routes
    projects/                         # Projects CRUD
    meetings/                         # Meetings rooms logic
    chat/                             # RAG assistant endpoints
  services/                           # Internal business logic domains
    identity/                         # SuperTokens auth verification
    projects/                         # Project permission checks
    meetings/                         # LiveKit room manager
    knowledge/                        # Semantic analysis
    ai/                               # LLM wrapper & Prompt management
  db/                                 # DB Layer
    models/                           # SQLAlchemy schemas
    repositories/                     # Postgres queries / pgvector wrappers
  workers/                            # Background event stream consumers
    transcript/
    embedding/
  websocket/                          # Real-time WebSocket hubs
  main.py                             # FastAPI Application entrypoint
```

6.2 Backend-for-Frontend (BFF) Layer

The Gateway API acts as a BFF layer. The frontend never executes multiple REST requests to collect dashboard data. Instead, composite endpoints aggregate information:

- GET /projects/{id}/overview - Aggregates project properties, current members list, latest meeting summaries, open action items, and recent timeline activity into one JSON payload.

- **GET /dashboard** - Consolidates all accessible user projects, upcoming schedules, and recent system notifications.

6.3 Execution Contracts: Layer Separation

Every domain features a clear, unidirectional responsibility model:

- **API Layer (Router):** Inspects HTTP parameters, authenticates headers via SuperTokens, calls the Service layer, and formats output.
- **Service Layer (Business Logic):** Implements permissions, controls transactions, schedules events, and coordinates internal modules.
- **Repository Layer (CRUD):** Focuses strictly on SQL queries, joins, filters, and paginations via SQLAlchemy. It is agnostic to business rules.

```

1 # db/repositories/project_repo.py
2 class ProjectRepository:
3     def __init__(self, db_session):
4         self.db = db_session
5
6     def get_by_id(self, project_id: str) -> ProjectModel:
7         return self.db.query(ProjectModel).filter(ProjectModel.id == project_id)
8         .first()
9
10 # services/project_service.py
11 class ProjectService:
12     def __init__(self, repo: ProjectRepository, auth_service):
13         self.repo = repo
14         self.auth = auth_service
15
16     def retrieve_project_details(self, project_id: str, user_id: str):
17         # 1. Enforce business rules & permissions
18         if not self.auth.has_permission(user_id, project_id, role="member"):
19             raise PermissionDeniedException("Insufficient workspace permissions")
20
21         # 2. Query database via Repository
22         project = self.repo.get_by_id(project_id)
23         if not project:
24             raise NotFoundException("Project not found")
25         return project

```

Listing 6.1: Clean Layer separation in FastAPI

Chapter 7

Roadmap and System Specs Blueprint

7.1 System Blueprints

To ensure structured implementation, the project relies on four specifications:

1. **API Contract:** Detailed OpenAPI JSON document defining parameters, schemas, and codes.
2. **Database Schema:** Migration SQL DDL containing table indexes, cascade behaviors, and foreign key definitions.
3. **Event Specification:** JSON structure definitions for Redis Stream events, documenting producers and consumers.
4. **UI-Backend Contract:** Matrix mapping screen views to REST paths, WS groups, and local store modifications.

7.2 Evolutionary Roadmap

The platform is scheduled to evolve through seven phases:

- **Phase 1: Collaboration foundation** - SuperTokens auth, LiveKit integration, basic CRUD for projects and meetings.
- **Phase 2: Transcription & Diarization** - Asynchronous faster-whisper pipeline, pyanote diarization, transcript storage.
- **Phase 3: Knowledge Synthesis** - LLM integration, extraction of facts, decisions, actions, and risks.
- **Phase 4: RAG Engine** - pgvector database integration, query translation, semantic retrieval with BGE-M3, and rerank pipelines.
- **Phase 5: Real-time Assistants** - Streaming chat APIs, live transcript updates over WebSockets.
- **Phase 6: Action Execution** - Task automation (Jira/GitHub), Google Calendar invites, and notifications.
- **Phase 7: Multi-Source Intelligence** - Ingesting wikis (Notion), code repositories (GitHub), and documents (PDFs) into the project memory.